

컴퓨터비전(AI응용)

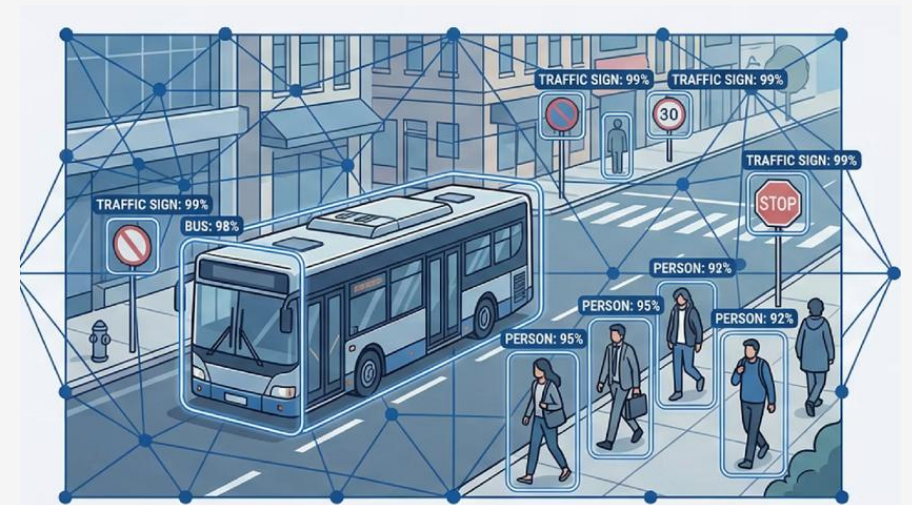
객체검출_2_One-stage Detector

◎ 메인 강의 목표

- ✓ YOLO v5 / v8 / v10의 공통점·차이점과 선택 기준 명확화
- ✓ Anchor 기반 → Anchor-Free → NMS-Free로의 진화 기술 이해
- ✓ 3줄 실습 코드로 학습부터 추론, 평가까지 파이프라인 구축

≡ 학습 로드맵

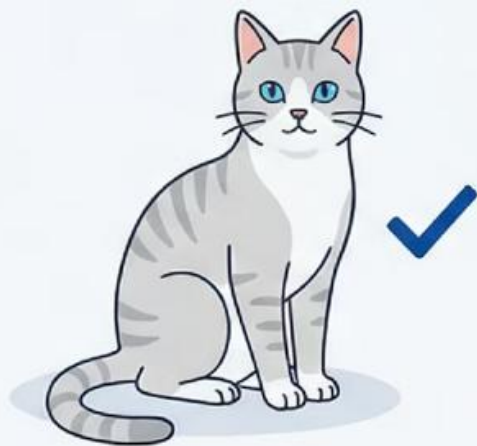
- **1단계: 원리 복습**
One-Stage vs Two-Stage, YOLO 계보
- **2~4단계: 버전별 심층 분석**
v5 (실전성) → v8 (앵커프리) → v10 (NMS-Free)
- **5단계: 비교 및 선택**
성능 지표 비교표, 산업별 가이드



‘AI가 세상을 보는 두 가지 방법’

분류 (Classification)

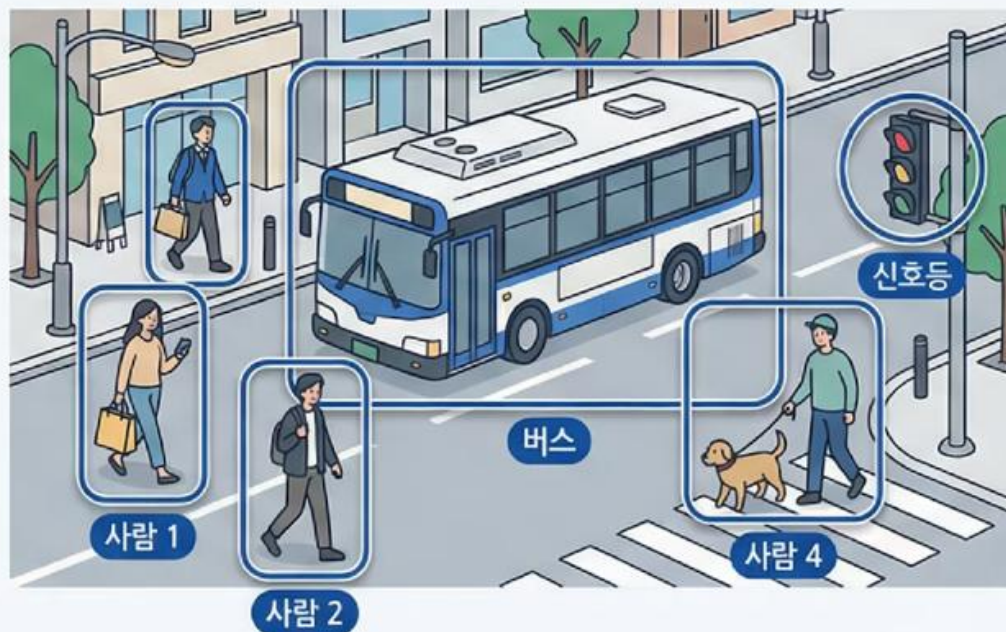
분류는 ‘무엇’인지만 답하지만,
검출은 ‘무엇’이 ‘어디에’ 있는지를 모두 찾아냅니다.



고양이

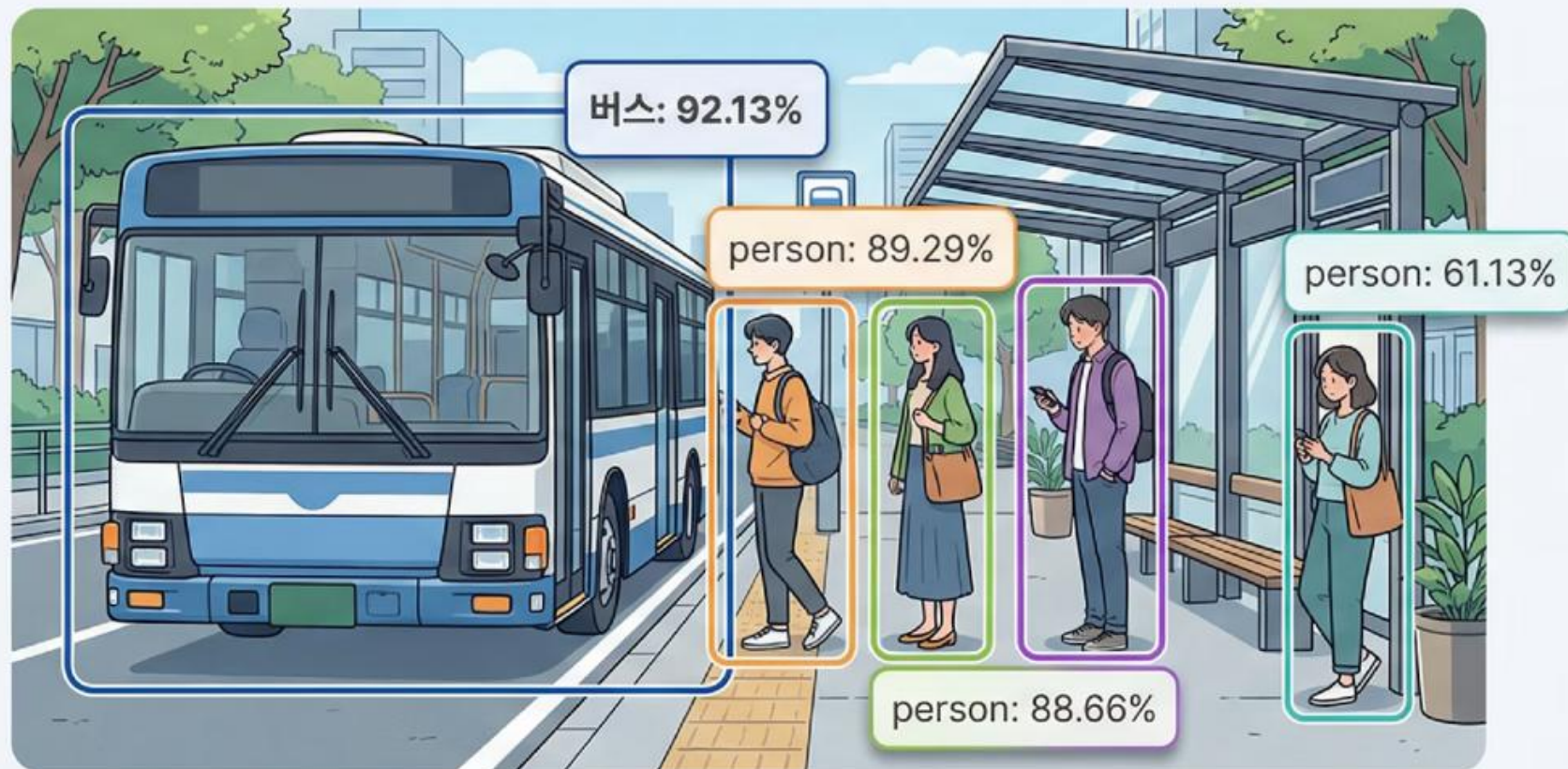
검출 (Detection)

하나의 이미지에서 버스 1개와 사람 4명을
동시에 발견하고 각각의 정확한 위치를 표시합니다.



92.13% 확률로 발견된 버스

실제 YOLOv8 추론 결과를 보여주는 화면 - 버스 정거장 장면에서 색색의 바운딩 박스와 신뢰도 수치들이 표시된 생생한 검출 결과

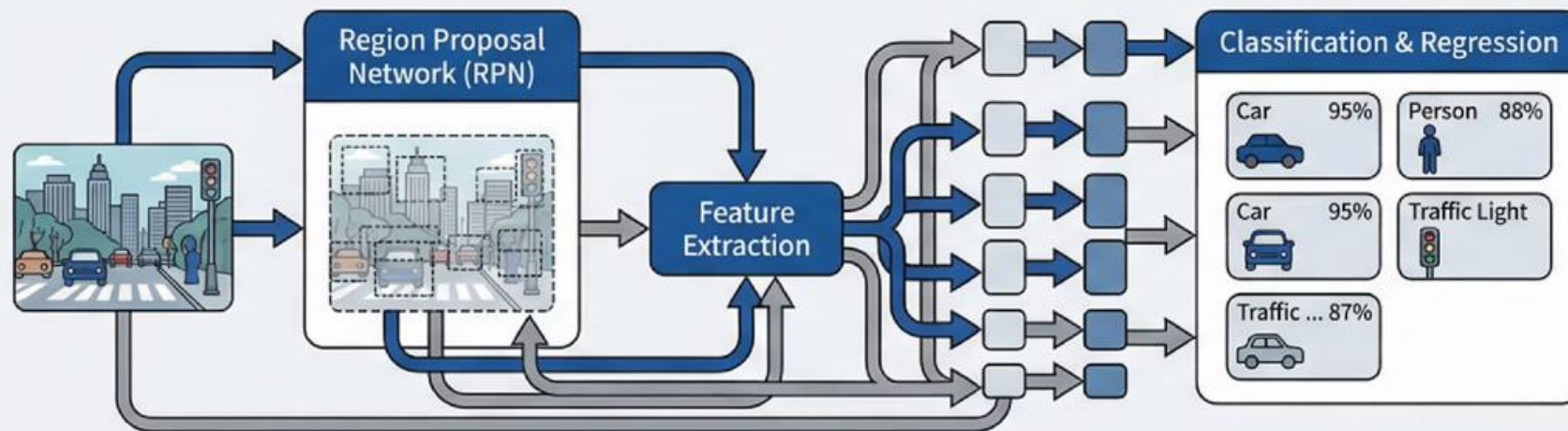


- 이미지 분류
 - 이미지 전체가 '**무엇**'인지 판별합니다.
 - 예를 들어 "이 사진은 고양이입니다"라고 답합니다.
- 객체 검출
 - 이미지 속 모든 객체의 '**위치**'와 '**종류**'를 찾아 표시합니다.
 - "50% 확률로 이 위치에 사람, 92% 확률로 이 위치에 버스"처럼 정확한 좌표와 신뢰도를 제공합니다.

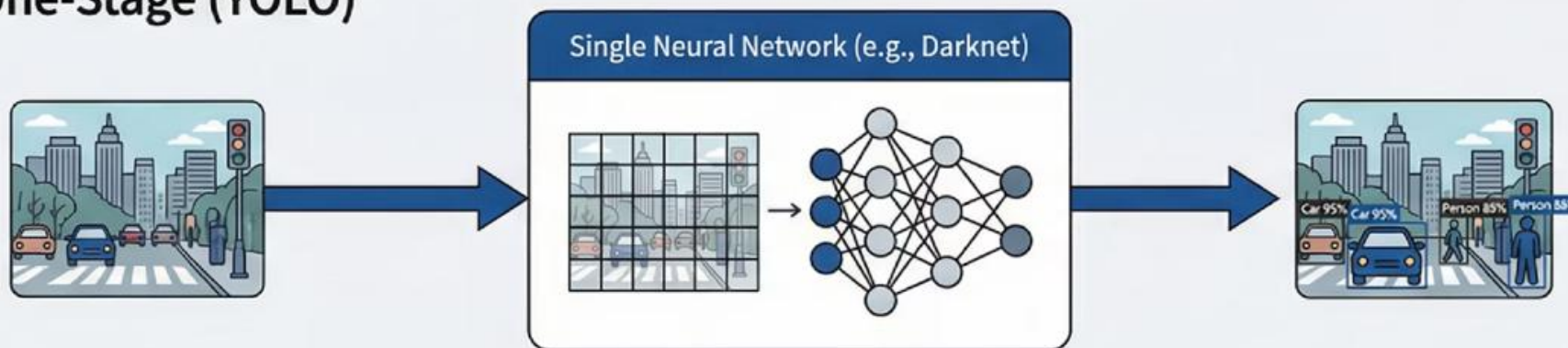
'YOLO의 혁신: 한 번에 모든 것을 본다'

Two-Stage (기존 방식)

기존 방식은 2단계로 나누어 처리했지만, YOLO는 단 한 번만 보고 모든 객체를 찾아냅니다



One-Stage (YOLO)



실시간 처리가 가능한 이유는 바로 이 One-Stage 방식 때문입니다

Two-Stage vs One-Stage Detector 비교

객체 검출의 두 가지 접근 방식: 정확도 중심의 2단계 vs 속도 중심의 1단계

Two-Stage Detector

정확도 우선



Faster R-CNN, Mask R-CNN

파이프라인

1. 영역 제안 (Region Proposal)
2. 분류 및 회귀 (Classification & Regression)

장점 높은 정확도 (특히 소형 객체)

단점 느린 속도 (실시간 어려움)

사용처 의료 영상, 정밀 검사, 서버 분석



비유

드라이브스루

주문 창구와 수령 창구가 분리됨 (정확하지만 이동 시간 소요)

One-Stage Detector

속도 우선



YOLO Series, SSD

파이프라인

단일 신경망이 이미지 전체를 한 번만(Once) 보며 위치와 클래스를 동시 예측

장점 매우 빠른 속도 (실시간 처리 가능)

단점 정확도 트레이드오프 (과거 이슈)

사용처 자율주행, CCTV, 모바일 앱, 로봇

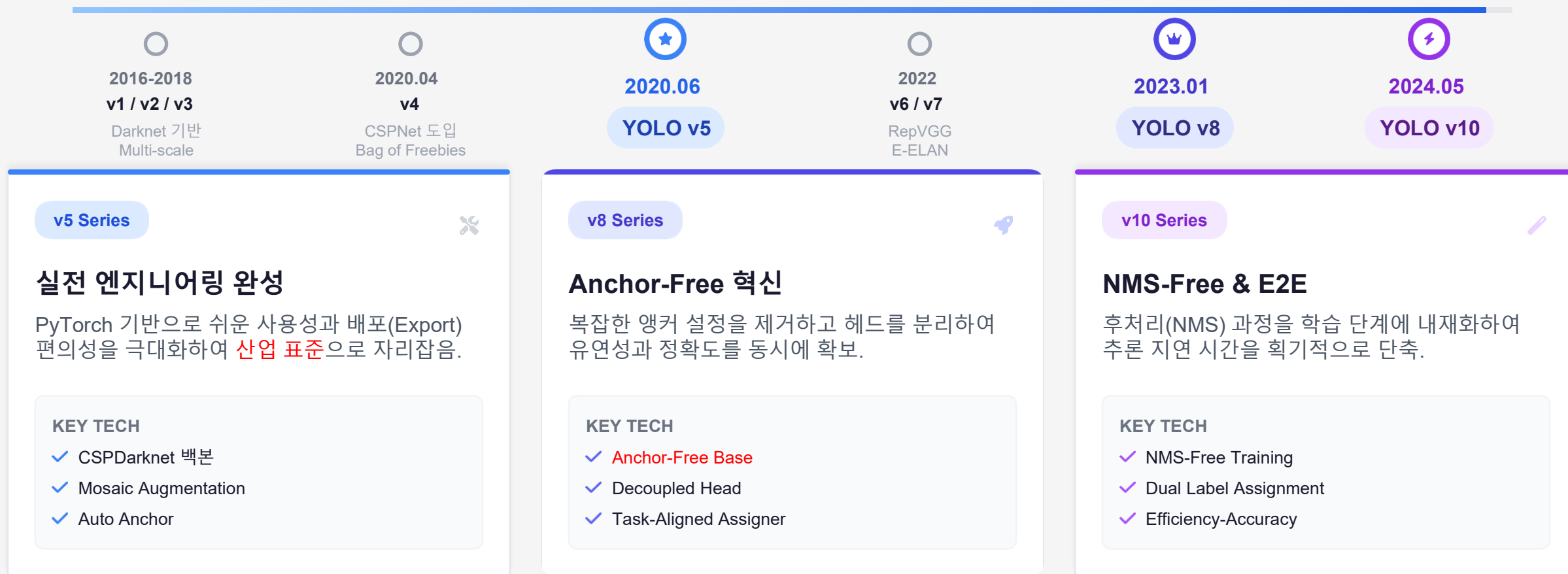


비유

편의점 계산대

물건을 올리는 즉시 바코드를 찍어 결제 (한 번에 처리)

One-Stage 방식은 단일 통합 예측으로 '무엇(Class) + 어디(Box)'를 한 번의 전방 패스(Forward Pass)에서 해결합니다.



YOLO는 '속도 유지'라는 대원칙 하에 '정확도(v5) → 구조적 유연함(v8) → 효율적 종단학습(v10)'으로 진화했습니다.

CSPDarknet53 & 아키텍처

v5는 **CSP(Cross Stage Partial)** 구조를 도입해 연산량은 줄이고 학습 능력(Gradient Flow)은 강화했습니다.

🧩 CSPNet 핵심 원리

✔ Split & Merge (분할과 병합)

입력 특징맵을 두 파트로 나누어, 하나는 연산(Conv)을 거치고 다른 하나는 우회하여 나중에 합칩니다.

✔ Gradient 중복 방지

역전파 시 중복되는 기울기 정보를 줄여 학습 효율을 높입니다.

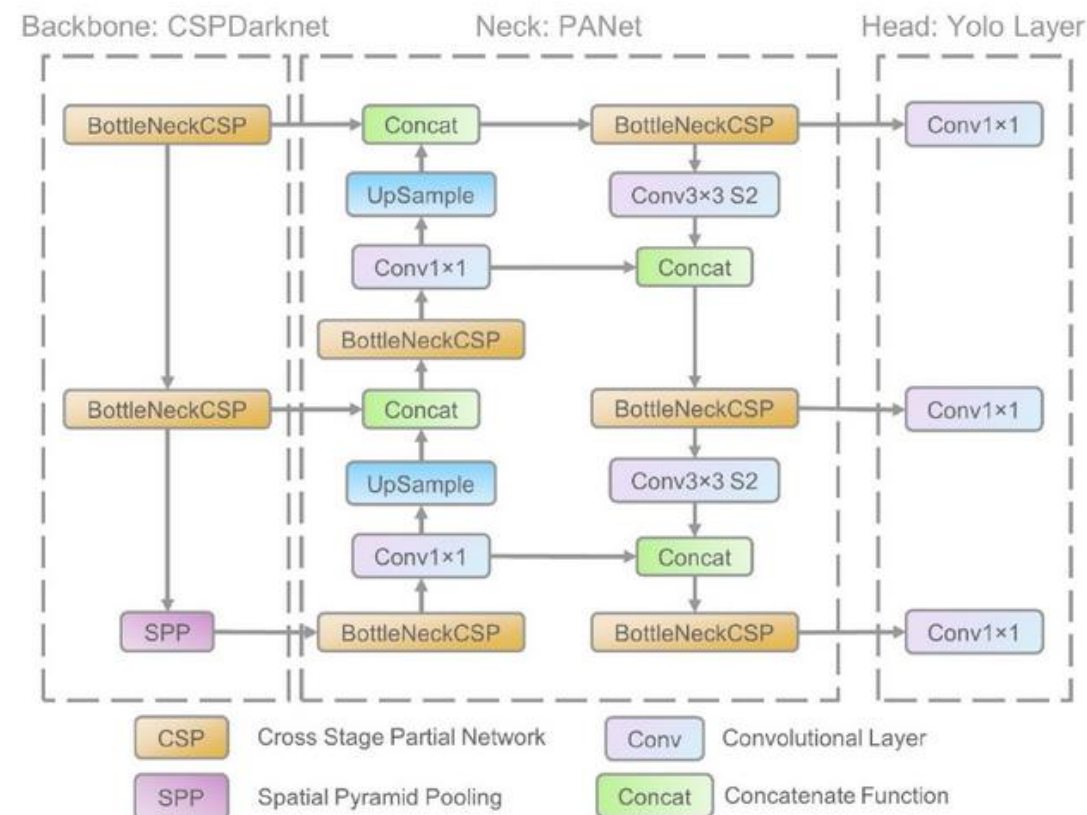


💡 핵심 비유: 고속도로 분기 차선

"정보가 두 갈래로 나뉘어 흐른다"

꼭 막힌 1차선 도로 대신, 본선(연산)과 추월차선(우회)으로 나누어 달린 후 합류하면 흐름(학습)이 훨씬 원활해집니다.

Architecture Overview



v5는 CSP(Backbone)로 연산 효율을 높이고, PANet(Neck)으로 다양한 크기의 물체를 놓치지 않고 포착합니다.

PANet (Path Aggregation Network)

백본에서 추출된 특징(Feature)을 융합하는 단계입니다.
Top-down(FPN)과 **Bottom-up(PAN)** 경로를 결합해
 정보 흐름을 양방향으로 만듭니다.

다중 스케일(Multi-Scale) 전략

P3

소형 객체 (Small)

80x80 grid / 얇은 특징 / 작은 물체 검출

P4

중형 객체 (Medium)

40x40 grid / 중간 특징 / 일반 물체 검출

P5

대형 객체 (Large)

20x20 grid / 깊은 특징 / 큰 물체 검출



💡 핵심 원리: 양방향 정보 고속도로

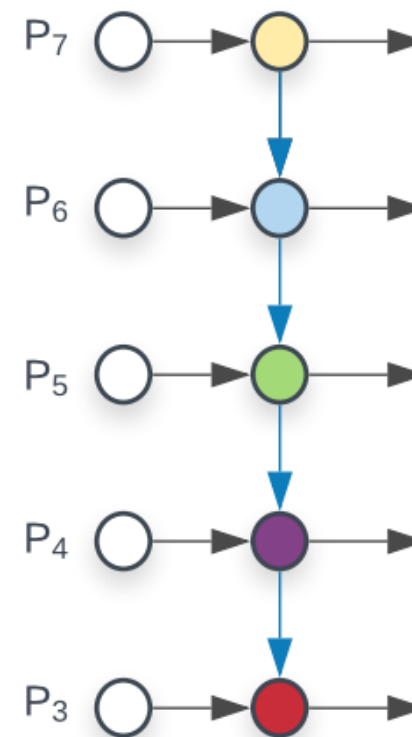
"의미는 내리고, 위치는 올린다"

FPN(↓)

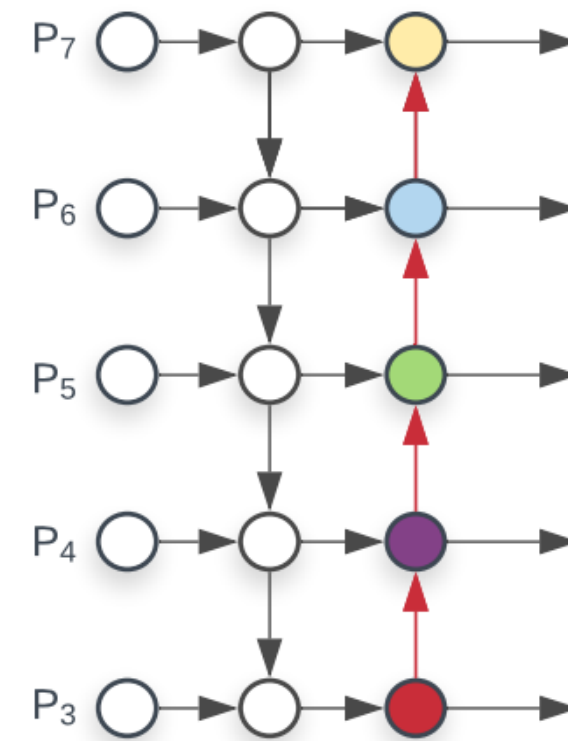
'무엇인지'에 대한 의미 정보를 아래로 전파하고,

PAN(↑)

'어디에 있는지'에 대한 위치 정보를 위로 다시 끌어올려
 정확도를 높입니다.



(a) FPN



(b) PANet

PANet은 상·하향 경로 결합으로 저해상도의 '의미'와 고해상도의 '위치' 단서를 모두 잡아냅니다

학습 데이터 최적화: 자동 앵커와 모자이크 증강

데이터셋의 특성을 스스로 학습하고 다양성을 극대화하는 v5의 핵심 전략

🔗 Auto Anchor (자동 앵커)

K-means 자동 산출

학습 시작 전, 데이터셋 내 모든 정답 박스(Bbox)의 크기 분포를 분석합니다.

📌 핵심 원리

K-means 클러스터링을 실행하여
내 데이터에 가장 적합한
초기 앵커 박스(Width, Height) 세트를 자동으로 생성

→ 사람이 직접 설정할 필요 없음

🗪 Mosaic Augmentation

4장 → 1장 합성의 마법

서로 다른 4장의 학습 이미지를 무작위로 크롭하고 배치하여 하나의 학습 이미지로 합성합니다.



- ✓ 소형 객체 검출력 향상
- ✓ 배치 크기 의존도 감소

마치 퍼즐 조각을 섞어 맞추듯,
다양한 배경과 맥락을 강제로 학습시켜
새로운 장면(일반화)에 더 강해집니다.

적은 데이터로도 높은 성능 확보 가능

Auto Anchor는 데이터 적합성을, Mosaic은 배경 다양성을 동시에 끌어올립니다.

v5 모델 크기 라인업 (n/s/m/l/x)

용도와 하드웨어 제약에 따라 최적의 모델 크기를 선택하는 것이 실무의 핵심

모델 성능 비교표 (COCO Val)

Model	Size	mAP _{val} (0.5:0.95)	Speed CPU b1 (ms)	Params (M)	FLOPs @640 (B)
YOLOv5n	Nano	28.0	45	1.9	4.5
YOLOv5s	Small	37.4	98	7.2	16.5
YOLOv5m	Medium	45.4	224	21.2	49.0
YOLOv5l	Large	49.0	430	46.5	109.1
YOLOv5x	XLarge	50.7	766	86.7	205.7

i n → x: 정확도(mAP) 증가, 속도(Speed) 감소

* b1: batch size 1

실무 선택 가이드



모바일 / 엣지 (Edge)

추천: v5n / v5s

지연 시간(Latency)에 민감하고 전력/메모리 제약이 심한 환경.
라즈베리 파이, 모바일 앱, 드론 등.



범용 서버 / PC

추천: v5m / v5l

GPU가 있는 일반적인 서버 환경.
정확도와 속도의 균형이 중요할 때 가장 무난한 선택.



고정밀 / 연구용

추천: v5x

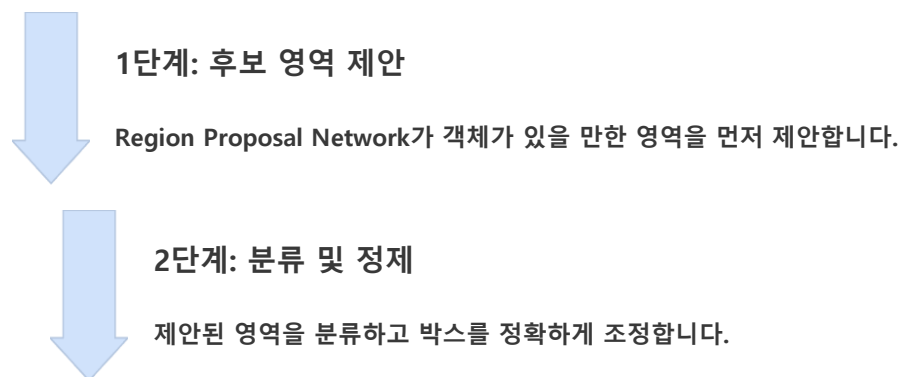
실시간성보다 0.1%의 정확도 향상이 중요한 경우.
의료 영상 분석, 정밀 제조 결함 검출 등.

용도와 제약 사항(하드웨어, 지연 시간)에 맞춰 n(경량/고속)부터 x(고성능/저속)까지 정밀도-속도 균형을 선택합니다.

YOLOv8을 활용한 Object Detection

Two-Stage Detector

예시: Faster R-CNN



높은 정확도, 느린 속도

One-Stage Detector

예시: YOLO (You Only Look Once)

단일 단계 예측

이미지를 한 번만 보고 위치와 분류를 동시에 예측합니다.

실시간 처리 가능, 우수한 정확도

YOLOv8의 주요 특징



Anchor-Free 설계

미리 정의된 앵커 박스 없이 객체의 중심점과 크기를 직접 예측하여 모델이 더 유연하고 단순해졌습니다.



통합 프레임워크

객체 검출, Segmentation(영역 분할), Classification(분류), Pose Estimation(자세 추정)을 하나의 프레임워크로 처리 가능합니다.



다양한 모델 크기

n(nano)부터 x(xlarge)까지 제공됩니다.
작은 모델(n, 3.2M 파라미터)은 빠르고,
큰 모델(x, 68.2M 파라미터)은 정확도가 높습니다.

YOLOv8: v5와 무엇이 다른가?

가장 큰 변화는 Anchor-Based에서 **Anchor-Free**로의 패러다임 전환입니다.

🔄 핵심 변화: Anchor-Free로의 전환

기존 문제점 (Anchor-Based)

사전 정의된 앵커 박스 크기에 의존하여, 최적의 앵커를 찾기 위한 K-means 등 추가 연산과 하이퍼파라미터 튜닝이 필수적임.



중심점(Center-based) 예측

객체의 중심점과 중심으로부터의 거리(Top, Bottom, Left, Right)를 직접 예측합니다.



Decoupled Head (분리형 헤드)

분류(Classification)와 위치 회귀(Regression) 브랜치를 분리하여 학습 간섭을 줄입니다.

★ 왜 중요한가?

앵커 튜닝 비용이 0이 되며, 비정형 비율의 객체나 극소형 물체 검출에서 훨씬 유연한 성능을 발휘합니다.

🏠 비유 1: 위치 찾기 방식

Anchor-Based



지도 격자

"가장 가까운 칸을 찾아라"

Anchor-Free (v8)



GPS 좌표

"정확한 위도/경도 직송"

격자에 구애받지 않고 객체가 있는 곳을 바로 찍습니다.

👕 비유 2: 옷 맞추기

Anchor



기성복

"S, M, L 중 골라 수선"

Anchor-Free



맞춤복

"치수 재서 바로 제작"

초기 설정값(기성복)에 얽매이지 않고 최적의 박스를 만듭니다.

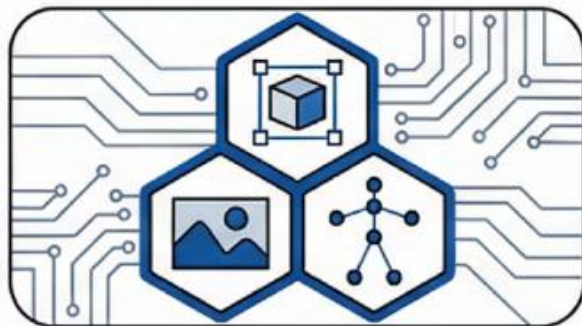
v8은 앵커(초기 박스) 없이 '좌표를 직접 예측'하여 모델의 유연성과 일반화 성능을 동시에 잡았습니다.

YOLOv8의 3가지 핵심 혁신



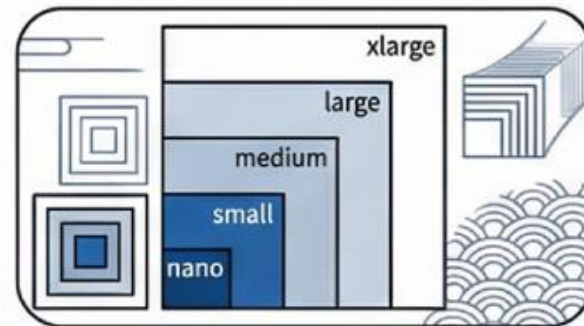
1. Anchor-Free 설계

앵커 박스 없이도 정확한 예측이 가능한
Anchor-Free 설계로 모델이
더욱 유연해졌습니다



2. 통합 프레임워크

객체 검출, 영역 분할, 자세 추정을
하나의 통합 프레임워크에서 처리합니다



3. 다양한 모델 크기

nano부터 xlarge까지, 상황에 맞는
최적의 모델 크기를 선택할 수 있습니다



C2f (Cross Stage Partial + Feature Flow)

v8의 핵심 빌딩 블록인 C2f는 **더 풍부한 Gradient Flow**를 위해 설계되었습니다. 일부 채널만 잔차(Residual) 경로로 우회시켜 계산은 줄이고 표현력은 유지합니다.

✓ 직관적 수식 해설

1. 기존 Residual (ResNet/v5 C3)

입력과 변환 결과를 단순히 더합니다.

$$y = x + F(x)$$

2. C2f 모듈 (v8)

입력을 분할(Split)하고, 각 단계의 결과를 모두 합칩니다(Concat).

$$y = \text{Concat}(x_1, \text{Block}(x_2), \dots)$$

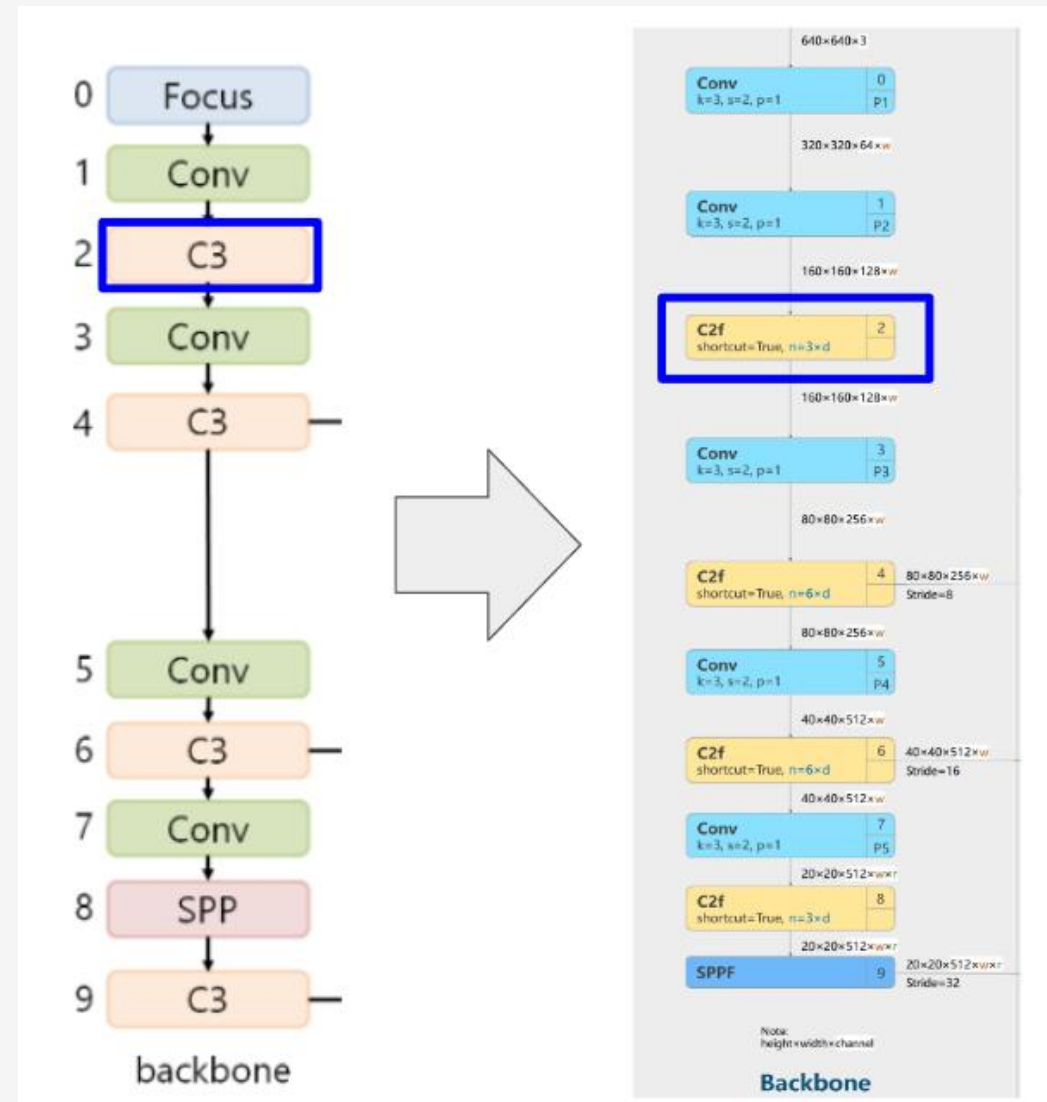


💡 **핵심 비유: 고속도로 분기 차선**

"정보가 여러 갈래로 나뉘어 흐른다"

딱 막힌 1차선 대신, 도로를 **여러 개의 분기 차선**으로 쪼갬 뒤(Split), 각 차선에서 처리된 차들이 **한 번에 다시 합류**(Concat)하여 정체 없이 풍부한 정보를 전달합니다.

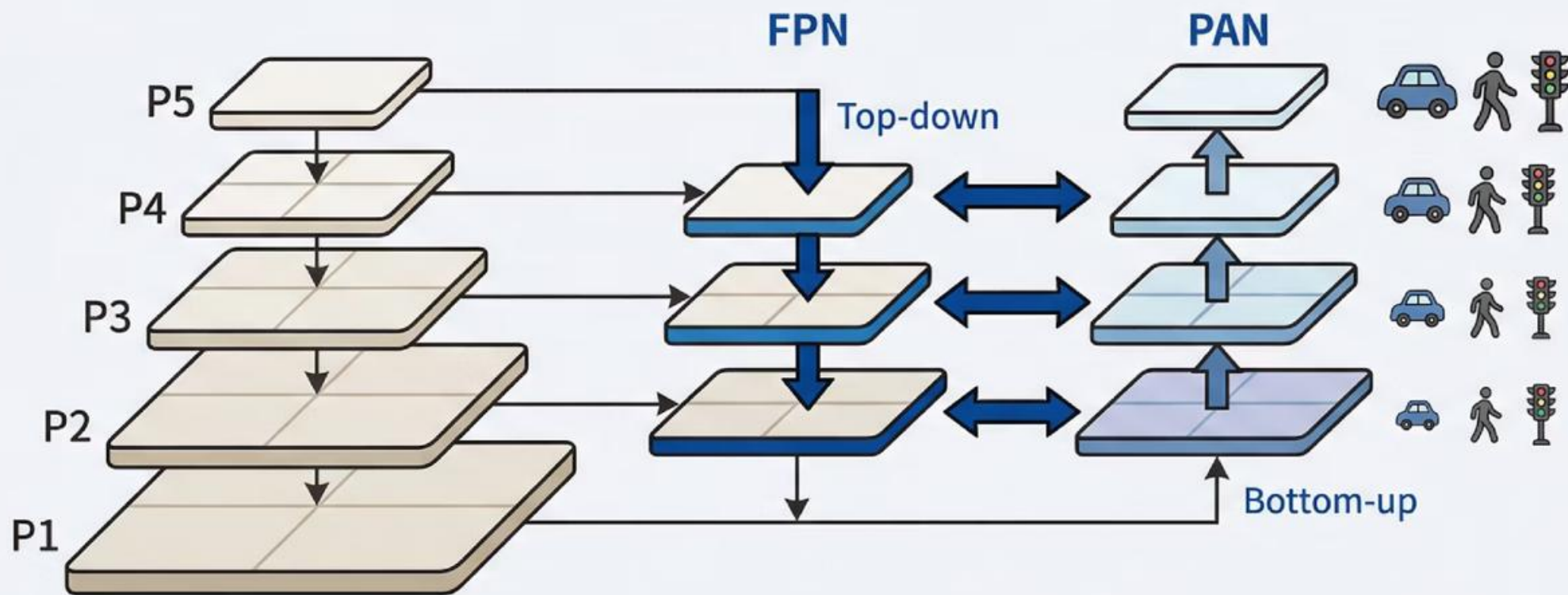
C2f Module Architecture



C2f는 '부분 분기(Split) + 재합류(Concat)' 구조로 연산 효율과 정보 보존(Gradient Flow)을 동시에 확보합니다.

작은 객체를 놓치지 않는 비밀: 다중 스케일

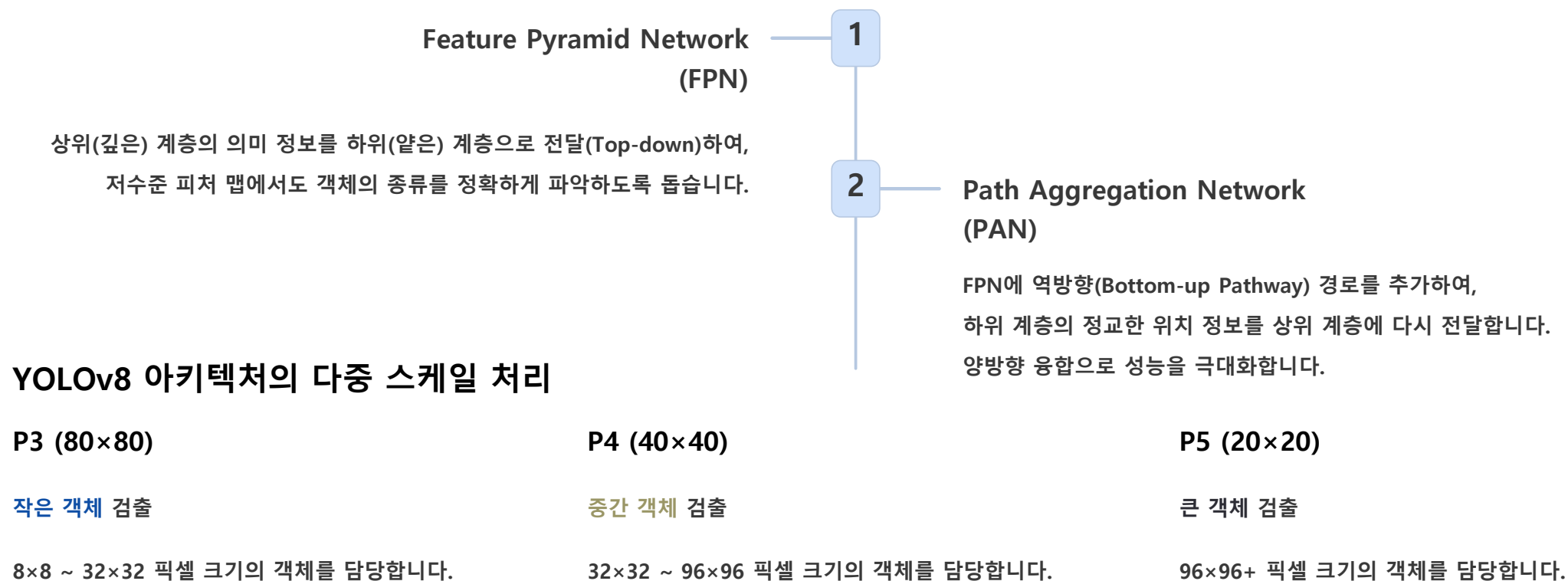
깊은 신경망은 의미는 잘 파악하지만 작은 객체의 위치 정보를 잃어버리는 문제가 있습니다



FPN은 상위 계층의 의미 정보를 하위로, PAN은 하위 계층의 위치 정보를 상위로 전달합니다

FPN과 PAN의 원리

❏ 핵심 문제: 깊은 신경망은 추상적인 의미(Semantic) 정보는 강하지만, 작은 객체의 위치(Spatial) 정보는 손실됩니다.



Backbone(CSPDarknet)에서 특징을 추출한 후, Neck(PAN-FPN)을 통해 다양한 크기의 피쳐 맵(Feature Map)을 생성하여 모든 크기의 객체를 효과적으로 검출합니다.

PAN-FPN: 양방향 특징 융합

단일 방향 정보 전달의 한계를 넘어, **Top-down(의미)**과 **Bottom-up(위치)** 경로를 결합하여 모든 크기의 객체를 포착합니다.

3단 구조 비교

FPN ↓ 고수준 의미(Semantic)를 하위로 전달

PAN ↑ 위치 정보(Localization)를 상위로 전달

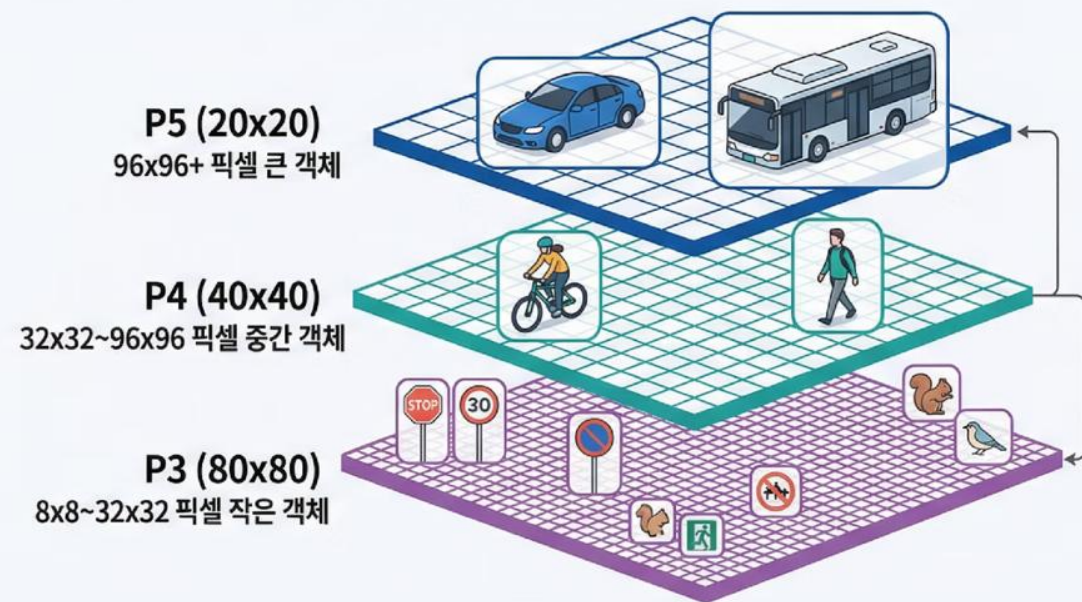
v8 Neck 양방향 결합 (FPN + PAN) + C2f 모듈

Feature Level별 담당 객체

P3 Small Objects (Stride 8)
작은 물체, 멀리 있는 사물 (고해상도 특징맵)

P4 Medium Objects (Stride 16)
중간 크기 물체, 일반적인 사물

P5 Large Objects (Stride 32)
화면을 가득 채우는 큰 물체 (저해상도, 강한 의미정보)



PAN-FPN은 의미(Semantic)와 위치(Spatial) 단서를 상하로 재순환시켜, 모든 스케일의 객체 검출 성능을 극대화합니다.

v8 Decoupled Head: 분리된 헤드의 이유

분류(Classification)와 회귀(Regression)의 충돌을 해결하기 위한 구조적 혁신

Coupled Head (기존)

v3 ~ v5

🔗 결합형 구조

구조적 특징

분류(Class)와 위치(Box) 예측이 동일한 컨볼루션 레이어 파라미터를 공유

문제점 **Task Misalignment (과제 충돌)**

영향 서로 다른 목표 함수가 경쟁하며 수렴 지연

결과 정확한 위치 선정과 클래스 분류 간 타협 발생



비유: 1인 2역

판사가 측량도 겸업

판결(분류) 내리면서 땅 넓이(회귀)도 직접 재려니 비효율적

Decoupled Head (v8)

최신 트렌드

🔗 분리형 구조

구조적 특징

분류 브랜치와 회귀 브랜치를 물리적으로 분리하여 독립적인 경로로 학습

장점 **학습 안정성 & 수렴 속도 가속**

효과 **Classification / Regression 간섭 제거**

결과 mAP 상승 및 예측 품질 향상



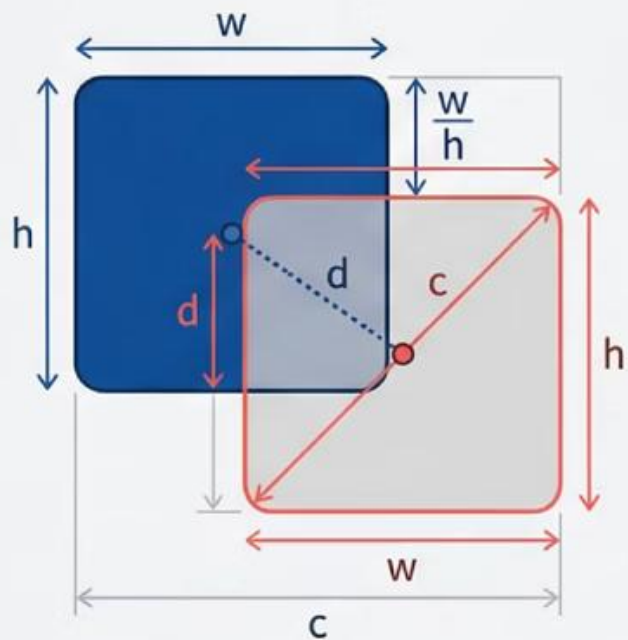
비유: 전문가 분업

판사와 측량사 분리 채용

판사는 판결(분류)만, 측량사는 위치(회귀)만 전담해 정확도 상승

헤드 분리(Decoupled Head)는 서로 다른 과제의 충돌을 해소해 학습 수렴을 돕고 추론 정확도를 높입니다.

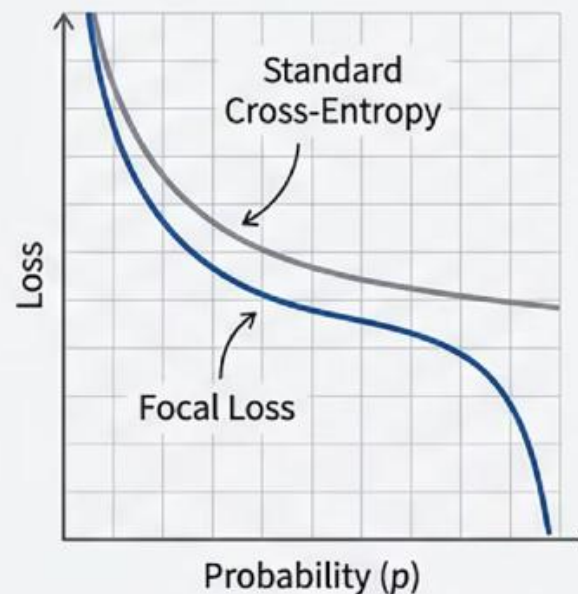
CloU Loss (Box Loss)



$$Loss = 1 - \text{CloU}$$

CloU Loss는 단순한 겹침뿐만 아니라
중심점 거리와 종횡비까지 모두 고려합니다

Focal Loss (Classification Loss)



$$FL = -\alpha(1-p)^\gamma \log(p)$$

Focal Loss는 쉬운 배경 샘플의 기여도를 줄이고
어려운 객체에 집중하여 학습합니다

모델이 학습할 때 '얼마나 틀렸는지'를 측정하는 손실 함수(Loss Function)는 객체 검출 성능의 핵심입니다. YOLOv8은 두 가지 주요 손실을 사용합니다.

Box Loss: 위치 오차 계산

목적: 예측 박스(pred)와 정답 박스(gt)의 위치 오차를 계산합니다.

$$\text{Loss} = 1 - \text{CIoU}$$

CIoU (Complete IoU)

단순히 겹치는 영역(IoU)뿐만 아니라, **중심점 거리**와 **종횡비**까지 고려하는 가장 정교한 손실 함수입니다. YOLOv8의 기본 Box Loss로 사용됩니다.

예를 들어,
두 박스가 절반만 겹치더라도 중심이 가까우면 더 나은 예측으로 평가합니다.

Classification Loss: 클래스 오차 계산

목적: 객체의 종류(클래스)를 얼마나 정확하게 예측했는지 계산합니다.

$$\text{FL} = -\alpha (1 - p)^{\gamma} \log(p)$$

Focal Loss (FL):

배경처럼 **쉬운 샘플**의 손실 기여도를 줄이고(*gamma* 파라미터 사용),
어려운 샘플에 집중하여 학습함으로써 클래스 불균형 문제를 해결합니다.

이미지 속 대부분은 배경이고 객체는 소수이므로,
어려운 객체 검출에 더 많은 학습 노력을 집중시킵니다.



Loss가 낮을수록 모델의 예측이 정답에 가까워집니다. 학습 과정에서 이 Loss 값들이 점점 감소하는 것을 확인할 수 있습니다.

v8 손실함수 및 할당 전략

"무엇(Class)"과 "어디(Box)"의 정합성을 높이기 위해 손실함수와 샘플 할당 방식을 정교화했습니다.

🎯 Task-Aligned Assigner

분류 점수와 위치 정확도(IoU)를 곱해 가장 좋은 후보를 동적으로 선택합니다.

Alignment Metric (정렬 점수)

$$t = s\alpha \times u\beta$$

t 통합 점수

s 분류 점수 (Score)

u 위치 정확도 (IoU)

α, β 가중치 (보통 0.5, 6.0)



CloU Loss (Box Regression)

$$L_{CloU} = 1 - IoU + \rho^2(b, b_{gt}) / c^2 + \alpha v$$

개념

단순 겹침(IoU)을 넘어 **중심 거리**와 **종횡비(비율)**까지 고려해 박스를 정밀하게 맞춤.

구성 요소

IoU 겹침 영역 (Overlap)

ρ 중심점 간 유클리드 거리

v 종횡비(Aspect Ratio) 일치도



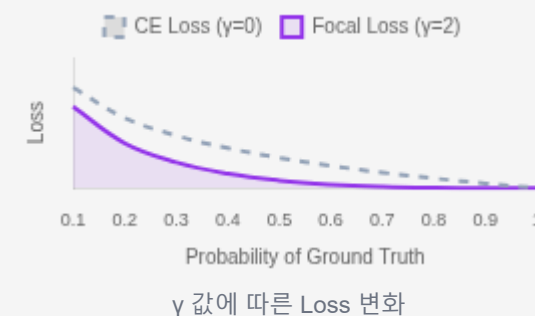
Focal Loss (Classification)

$$FL(pt) = -\alpha t(1 - pt)^{\gamma} \log(pt)$$

💡 핵심 비유

"쉬운 문제는 배점 낮게,
어려운 문제에 배점 집중"

이미 잘 맞추는 샘플(확률 높은 것)의 Loss 기여도를 0에 가깝게 낮춰,
모델이 틀린 문제에 집중하게 함.



CloU(위치 정밀도) + Focal(난이도별 가중) + 정렬 할당으로 검출의 정합성과 어려운 케이스 대응력을 동시에 높입니다.

평가지표 정의와 임계값 설정

mAP은 모델의 전체 성적표지만, 실제 서비스 운영 시에는 비즈니스 목표에 맞춰 Precision과 Recall의 균형을 선택해야 합니다.

Precision (정밀도)

오탐 최소화

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"모델이 예측한 것 중 실제 정답의 비율"
낮으면? → **허위 경보(오알람)**가 많아져 사용자가 피로해짐.

Recall (재현율)

미탐 최소화

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"실제 정답 중 모델이 찾은 비율"
낮으면? → 중요 객체를 **놓치는 사고(미검)** 발생.

mAP (Mean Average Precision)

종합 점수

mAP@0.5 IoU 0.5 이상을 정답으로 칠 때의 평균 점수

mAP@0.5:0.95 IoU 0.5~0.95 구간 평균 (더 엄격함)

Confidence Threshold 가이드



의료 영상 (암 진단)

Recall 우선

놓치면 생명 위험. Conf 0.1~0.3 등으로 낮게 설정하여 의심 부위 모두 검출.



보안 CCTV (침입)

Precision 우선

오알람(고양이/바람) 시 경찰 출동 비용 발생. Conf 0.6~0.8로 보수적 설정.



일반 (검색/분류)

Balanced

균형점 유지. Conf 0.4~0.5 (Default). F1-Score 최대화 지점 사용.

지표는 절대적 점수가 아닌 '운영 의사결정 도구'입니다. 리스크(오탐 vs 미탐)에 맞춰 Confidence 임계값을 조정하세요.

 메모리 · 속도 · 정확도의 트레이드오프


Python SDK Tip

파이썬 코드로 더 유연하게 제어할 수도 있습니다.

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
results = model.train(data='coco.yaml', epochs=100)
```

왜 중요한가?

복잡한 파이프라인 구축 없이 **단 한 줄**로 실험 가능

CLI와 Python SDK가 통일되어 있어 전환이 쉬움

재현성(Reproducibility) 높은 실험 환경 보장

v8은 경량(Nano)부터 고성능(X-Large) 라인업을 제공하며, 간결한 CLI 명령어로 실전 접근성을 극대화했습니다.

Safety Helmet 데이터셋 구조



데이터셋 다운로드

Kaggle에서 Safety Helmet 데이터셋을 다운로드합니다.
총 5000개의 이미지가 포함되어 있습니다.



클래스 확인

데이터셋에는 'head', 'helmet', 'person' 총 3개의 클래스가
포함되어 있음을 확인합니다.



어노테이션 변환

원본 데이터의 XML(.xml) 형식 라벨을
YOLOv8이 사용하는 .txt 형식으로 변환합니다.
각 파일에는 객체의 클래스와 좌표 정보가 포함됩니다.

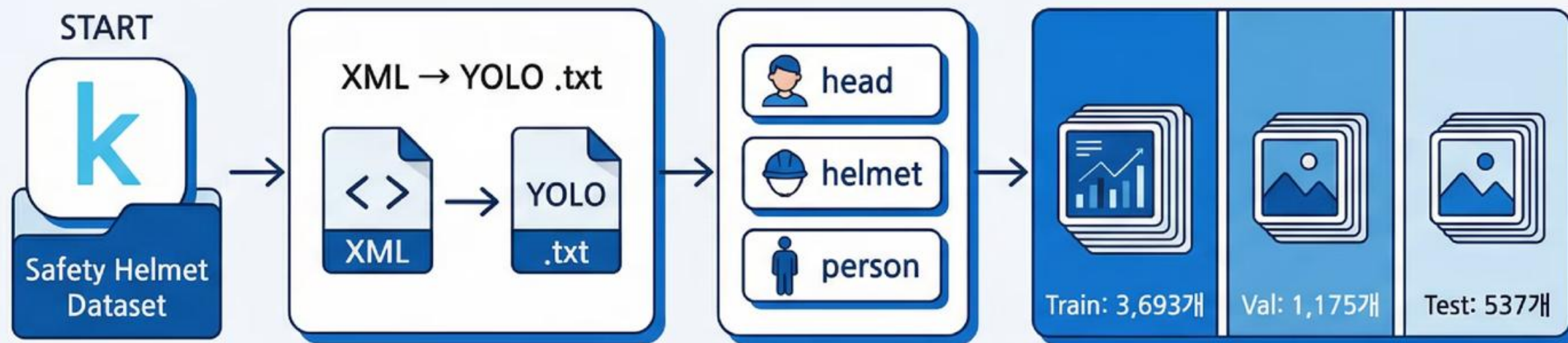


데이터 분할

전체 5000개 이미지를
Train(3693개), Validation(1175개), Test(537개)로 분할하여
YOLOv8 학습 형식에 맞춥니다.

Train 데이터로 모델을 학습하고, Validation 데이터로 성능을 모니터링하며, Test 데이터로 최종 성능을 평가합니다.

Kaggle의 Safety Helmet 데이터셋을 XML에서 YOLO 형식으로 변환합니다



Train 3693개, Validation 1175개, Test 537개로 과학적으로 분할합니다.
head, helmet, person 3개 클래스로 구성된 데이터셋과 변환 과정을 보여주는 화면

모자이크의 마법: 4장이 1장으로

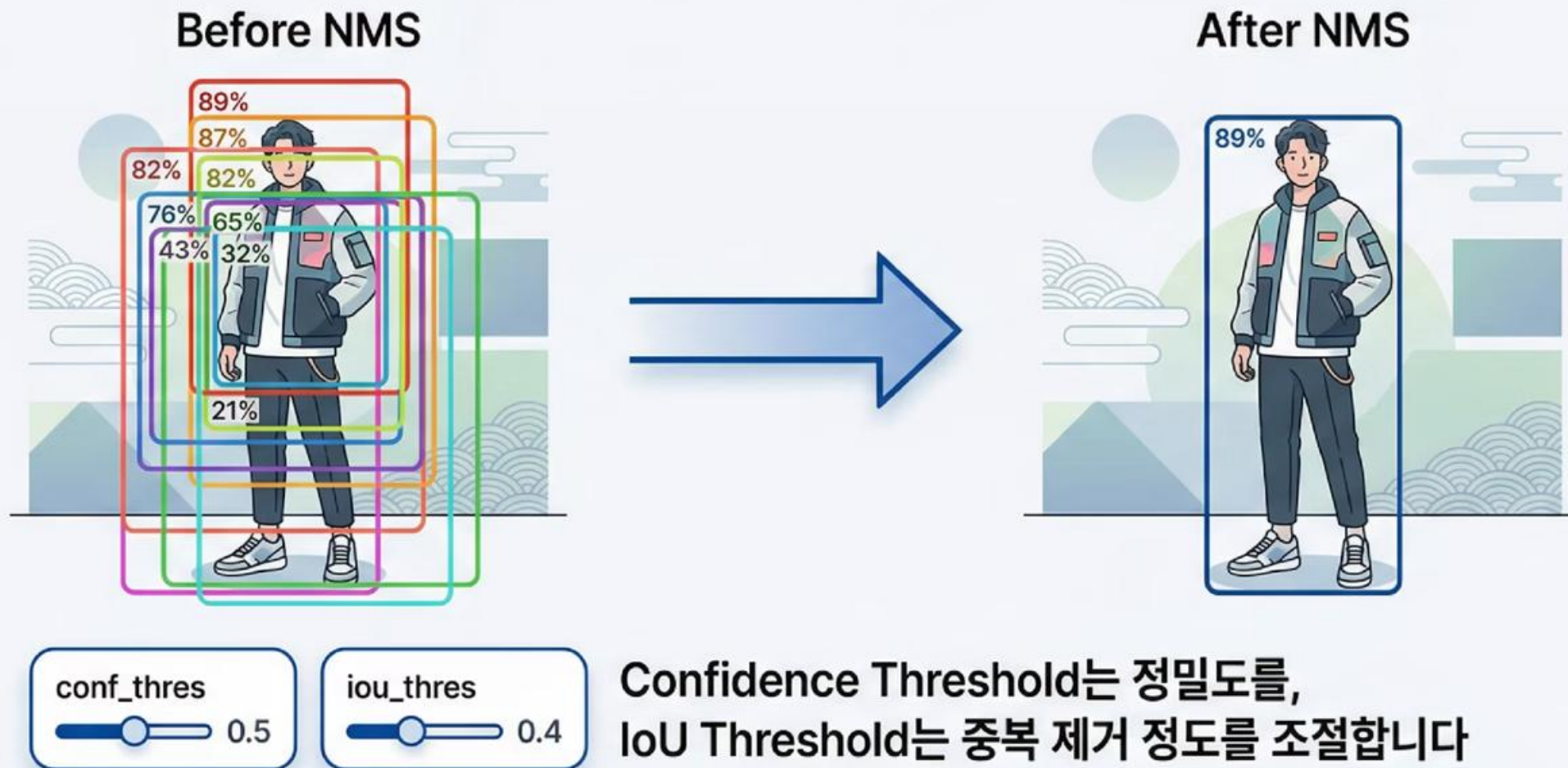
Mosaic Augmentation은 4장의 이미지를 퍼즐처럼 맞춰 새로운 학습 이미지를 생성합니다



다양한 크기의 객체들을 한 번에 학습하여
모델의 일반화 성능을 극대화합니다

NMS : 중복 제거 핵심 알고리즘

하나의 객체에 여러 개의 박스가 겹칠 때, 가장 확실한 하나만 남기고 나머지를 제거합니다



PyTorch 데이터 파이프라인



transforms: 이미지 전처리

Resize

224 × 224 픽셀로 크기 통일

ResNet18 입력 요구사항

ToTensor

이미지를 PyTorch 텐서로 변환

픽셀 값 범위: 0~1로 정규화

Normalize

평균과 표준편차로 표준화

ImageNet 통계 값 사용

☐ 편의점 알바 비유: Dataset은 진열대에서 물건을 하나씩 꺼내는 것이고, DataLoader는 그 물건들을 카트에 32개씩 담아서 한 번에 계산대로 가져오는 것입니다. 배치 크기(batch size)가 바로 카트의 크기입니다.

▶ Data Augmentation

- 데이터 증강(Data Augmentation)은 제한된 데이터셋으로도 모델의 일반화 성능을 크게 향상시키는 핵심 기법입니다.
- YOLOv8은 다양한 증강 기법을 자동으로 적용합니다.

Mosaic Augmentation

4장의 이미지를 모자이크처럼 이어 붙여 새로운 이미지를 생성합니다.
다양한 크기(Scale)의 객체를 한 번에 학습하여 일반화 성능을 높입니다.

Mixup

두 이미지를 섞어서(Blend) 새로운 이미지를 만드는 기법입니다.
모델이 더 부드러운 결정 경계를 학습하도록 돕습니다.

Copy-Paste

이미지 내 객체를 복사-붙여넣기하여 Hard Sample(어려운 샘플)을 인위적으로 생성합니다.
특히 작은 객체나 가려진 객체 검출 능력을 향상시킵니다.

▶ Strong Augmentation 파라미터

학습률(lr)과 모멘텀(momentum) 외에도 다양한 파라미터로 증강 강도를 조절할 수 있습니다.

hsv_h, hsv_s, hsv_v: 색상(Hue), 채도(Saturation), 명도(Value) 변형

translate: 이미지 이동 변환 scale: 크기 조절 degrees: 회전 각도

이러한 기하학적 및 색상 변형을 강하게 적용하여 모델의 강인함(Robustness)을 확보합니다.

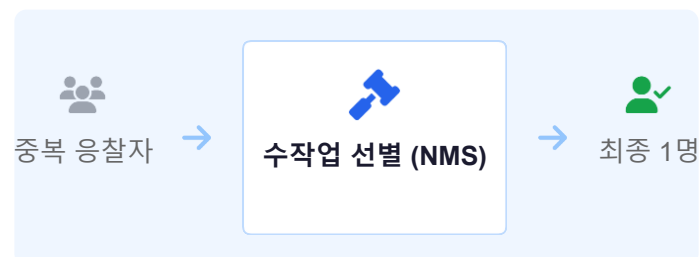
기존 NMS(비최대 억제)의 한계와 v10의 등장

객체 검출의 마지막 병목 구간이었던 후처리 과정을 어떻게 자동화했는지 알아봅니다.

⚠ 기존 NMS의 한계

- 민감한 하이퍼파라미터**
IoU 임계값(Threshold) 설정에 따라 검출 결과가 크게 달라짐 (오류 가능성↑)
- 추론 지연 시간 증가**
GPU 병렬 연산 후 별도의 순차적 연산이 필요해 전체 속도 저하
- 배포 복잡성**
TensorRT 등 최적화 시 NMS 플러그인 구현이 까다로움

💡 비유: 경매장 낙찰 시스템



"중복 응찰자 중 최고가 1명만 낙찰"

기존엔 수많은 중복 박스 중 점수가 가장 높은 1개를 사람이 정한 규칙(IoU)으로 골라냈습니다.
이는 느리고 부정확할 수 있습니다.

✓ v10의 혁신: NMS-Free

→ End-to-End 최적화

학습 단계에서 중복을 스스로 억제하도록 훈련시켜, 추론 시에는 후처리가 필요 없는 깔끔한 결과만 출력합니다.

기존 (v5/v8)	예측 → NMS → 결과
v10 (NMS-Free)	예측 → 결과 (즉시)

v10은 휴리스틱 후처리(NMS)를 학습 과정에 내재화하여 완전한 End-to-End 검출을 실현했습니다.

One-to-Many(예선) + One-to-One(본선)

v10은 학습 시 **두 가지 할당 방식**을 동시에 사용하여, 추론 단계에서 NMS(중복 제거) 과정을 완전히 없앴습니다.

Dual Assignment 원리

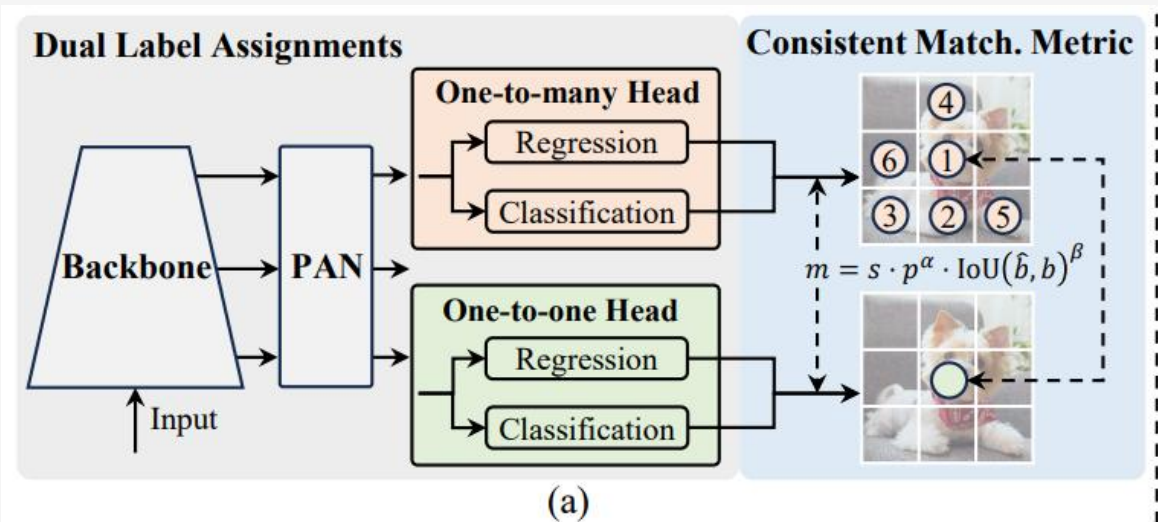
1 One-to-Many Head (예선)

하나의 정답(GT)에 여러 예측 박스를 할당하여 학습 신호를 풍부하게 제공합니다. (Recall 강화)

2 One-to-One Head (본선)

하나의 정답에 오직 하나의 예측 박스만 매칭시켜 중복을 억제합니다. (NMS 불필요)

Dual Assignment Architecture



💡 핵심 비유: 오디션 예선 vs 본선

"예선에선 많이 뽑고, 본선에선 1명 선발"

예선(Auxiliary)에선 재능 있는 후보들을 넉넉히 뽑아 훈련시키고(학습 효율↑), 최종 본선(Main)에선 우승자 1명만 무대에 세워(추론 깔끔) 중복을 없앱니다.

Dual Assignment는 중복 억제 과정을 학습 단계에 내재화하여, 추론 시 NMS 후처리 없이도 깔끔한 결과를 얻습니다.

효율성과 성능의 조화

v10은 NMS-Free뿐만 아니라 아키텍처 자체의 효율화를 통해
경량화와 성능을 동시에 달성했습니다

Large-kernel DWConv

대커널 심층별 컨볼루션

작은 커널(3x3)을 여러 번 쌓는 대신,
큰 커널(7x7 등)을 Depthwise로 적용하여
연산 효율을 높입니다.

핵심 원리

넓은 수용 영역(Receptive Field) 확보

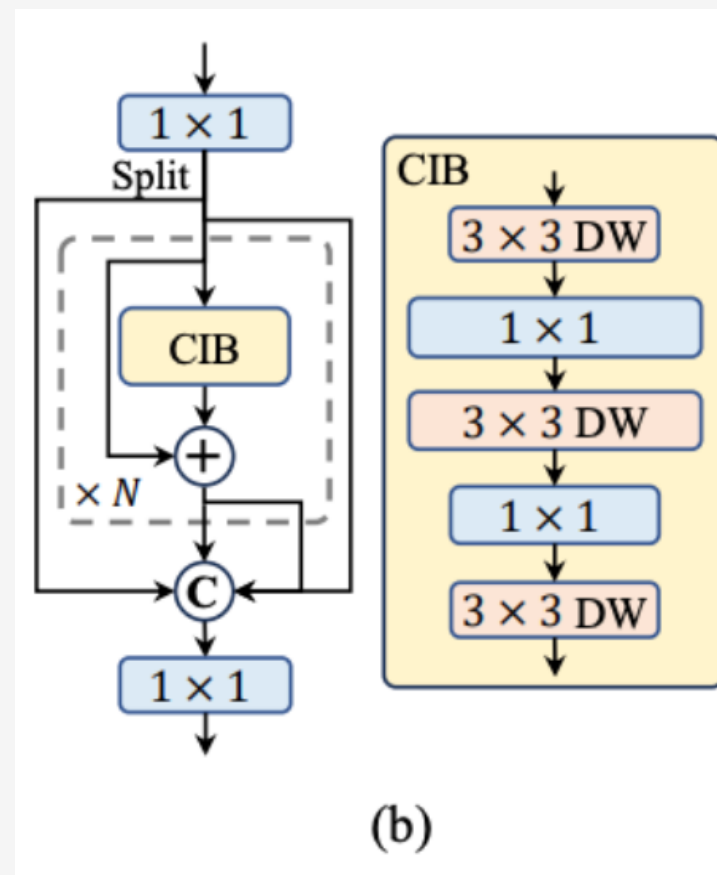
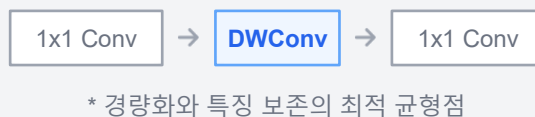
작은 물체와 큰 물체 동시 대응력 ↑

Depthwise 연산으로 파라미터 수 ↓

CIB 구조

Compact Inverted Block

"확장(Expand) → 깊이별(Depthwise) → 축소(Project)"
흐름을 통해 정보 손실을 최소화하며 연산량을 줄입니다.



후처리(NMS) 시간 단축 + 모델 추론 시간 단축
= 전체 지연시간(Latency) 극소화

모바일, 드론 등 연산 능력과 전력이 제한된 환경에서
최고의 성능 효율을 발휘합니다.

대커널 DWConv + CIB 구조 도입으로 '가볍고 넓게 보는' 특성을 구현하여 하드웨어 효율을 극대화합니다.

기존 CNN에서는 왜 3*3을 사용하였는가?

1. 연산량 절약

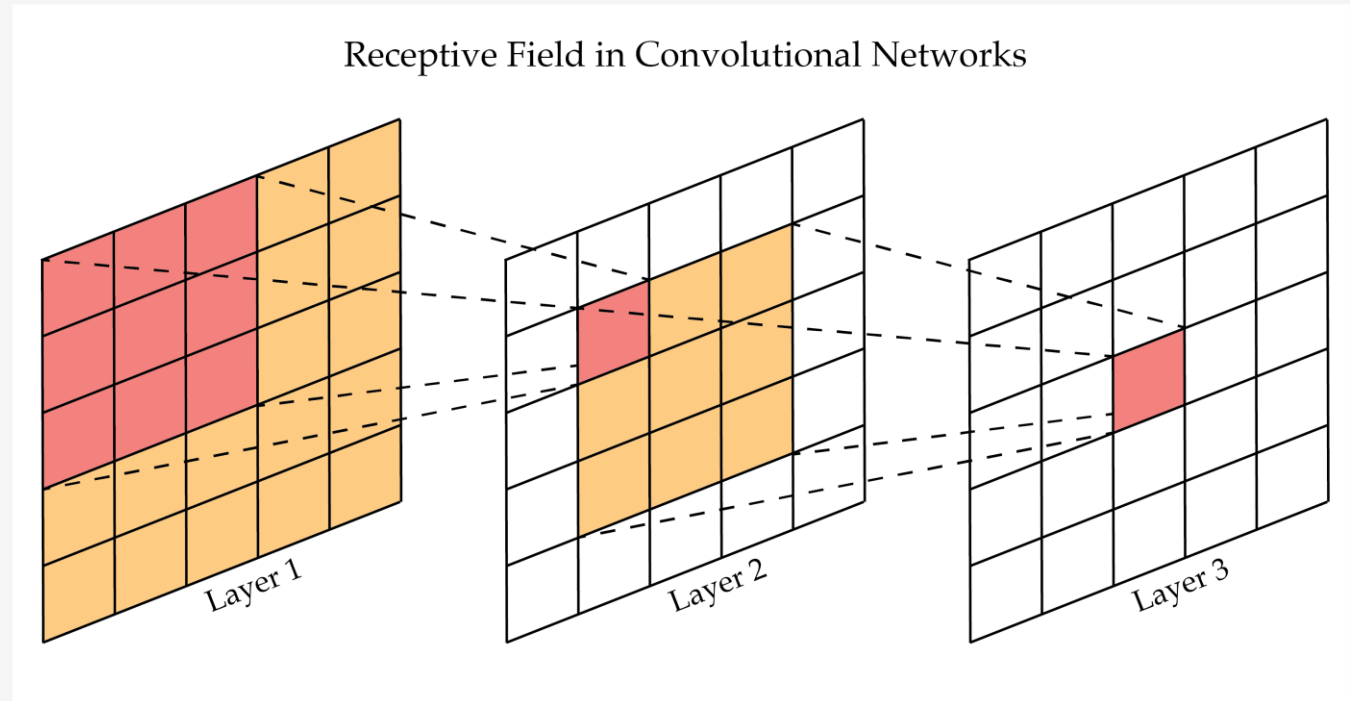
- $7*7 = 49$ 개 파라미터
- $3*3 = 9$ 개 파라미터 (훨씬 가벼움)

2. 비선형성 증가

- $3*3 + \text{Relu} + 3*3 + \text{Relu}$ (표현력 좋고)

3. 대체 가능

- $3*3$ 여러 개 쌓으면 $7*7$ 효과 가능 (receptive field 확대)



그런데 왜 Yolo v10은 7*7 을 쓰는 거야?

결론부터 말하면... Depth-wise Convolution 때문

- 일반 conv에서는 너무 무거워서 거의 안 씀

Depth-wise Convolution

일반 convolution은 모든 채널을 섞어서 계산

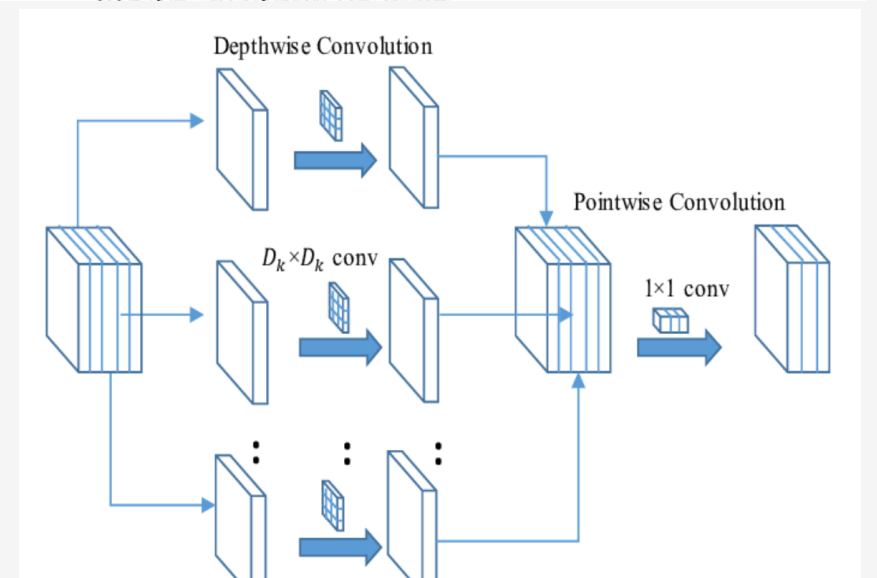
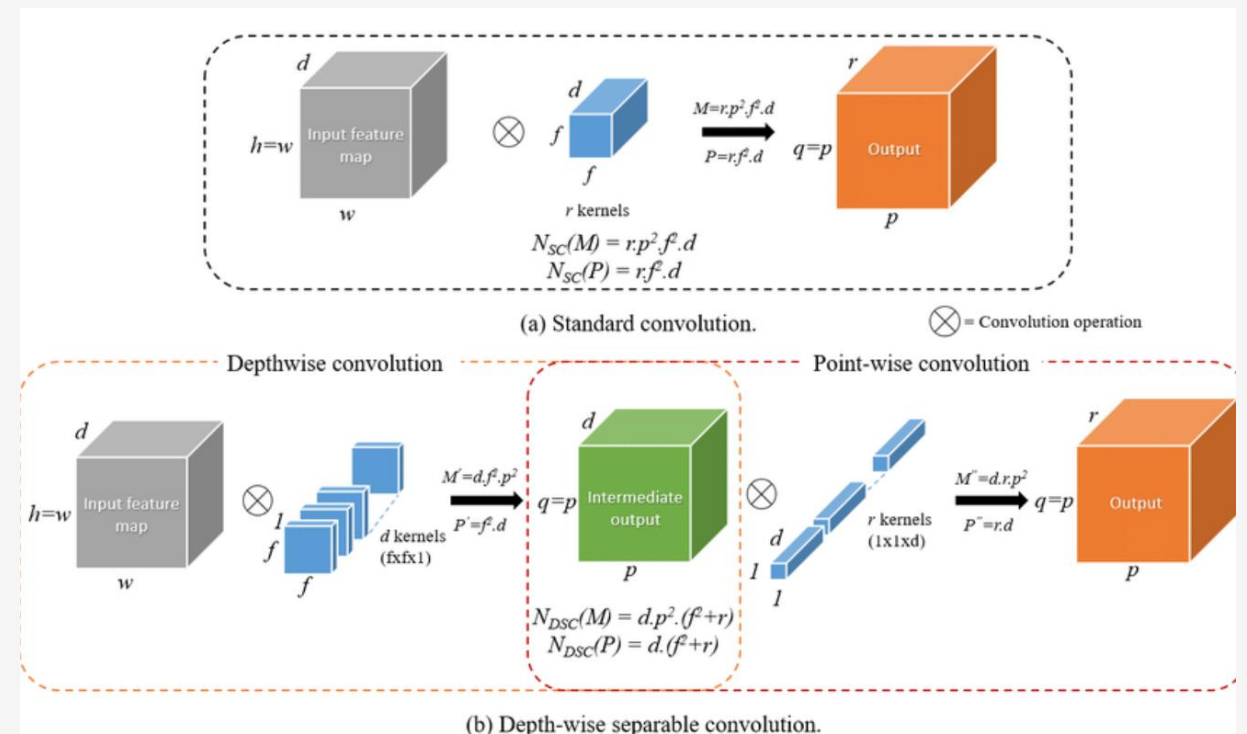
- 연산량이 큼

그러나, Depth-wise Convolution은
채널별로 따로 계산 (연산량 매우 작음)

직관적으로 이야기 해서,
3*3 여러 개 쓴다는 것은 작게 여러 번 본다는 의미
7*7 depth-wise 는 한 번에 넓게 보는 방식

Yolo v10 에서 7*7 쓰는 이유?

- 객체 탐지는 넓은 문맥(context) 중요
- depth-wise 덕분에 큰 커널 사용 가능하면서 연산량과 속도가 유지됨
- 즉, 적은 비용으로 큰 커널을 사용함으로써 더 넓은 정보를 볼 수 있게 되었음



YOLO v5, v8, v10 비교

아키텍처·성능·특징 비교를 통해 프로젝트에 최적화된 모델 선택하기

YOLO v5

안정성

아키텍처 CSP + PANet

앵커 방식 Anchor-Based

헤드 구조 Coupled Head

후처리 NMS 필수

손실함수 GloU / CloU

성능 지표 (v5s 기준)

mAP 37.4

Params 7.2M

추천 용도

레거시 호환, 튜닝 자유도

YOLO v8

표준/범용

아키텍처 C2f + PAN-FPN

앵커 방식 Anchor-Free

헤드 구조 Decoupled Head

후처리 NMS 필수

손실함수 CloU + Focal + DFL

성능 지표 (v8s 기준)

mAP 44.9

Params 11.1M

추천 용도

고성능, 쉬운 학습/배포

YOLO v10

초고속/효율

아키텍처 CIB + Partial SA

앵커 방식 Anchor-Free

헤드 구조 Decoupled Head

후처리 NMS-Free

손실함수 Dual Assignment

성능 지표 (v10s 기준)

mAP 46.3

Params 8.0M

추천 용도

엣지 디바이스, 최소 지연

안정성(v5), 고성능 표준(v8), 초저지연 엔드투엔드(v10) 중 프로젝트 제약사항(하드웨어, 데이터)에 맞춰 선택합니다.

산업별 YOLO 선택 가이드

자율주행, 의료, 보안, 엣지 등 각 도메인 특성과 제약사항에 따른 최적 모델 제안



자율주행 (Autonomous Driving)

권장 모델

v8 / v10 (Large/Medium)

- ✓ 복잡한 장면 대응: 앵커프리(v8) 및 NMS-Free(v10) 구조가 밀집된 객체 인식에 유리
- ✓ Latency 임계값: v10의 후처리 제거로 추론 지연 시간을 극소화하여 실시간성 확보



의료 영상 (Medical Imaging)

권장 모델

v8 (Large/X-Large)

- ✓ 정확도 최우선: 속도보다 False Negative(미검출) 방지가 핵심. 대형 모델 사용 권장
- ✓ 희귀 케이스: v8의 Focal Loss 등은 데이터 불균형(정상 vs 질병) 학습에 효과적



보안 CCTV (Surveillance)

권장 모델

v5 (Medium) 또는 v8 (Small)

- ✓ 안정성/가성비: 24시간 가동 서버 부하를 고려한 중소형 모델 선택
- ✓ 검증된 파이프라인: v5는 배포 안정성이 매우 높아 레거시 시스템 통합에 유리



모바일 & 엣지 (Mobile Edge)

권장 모델

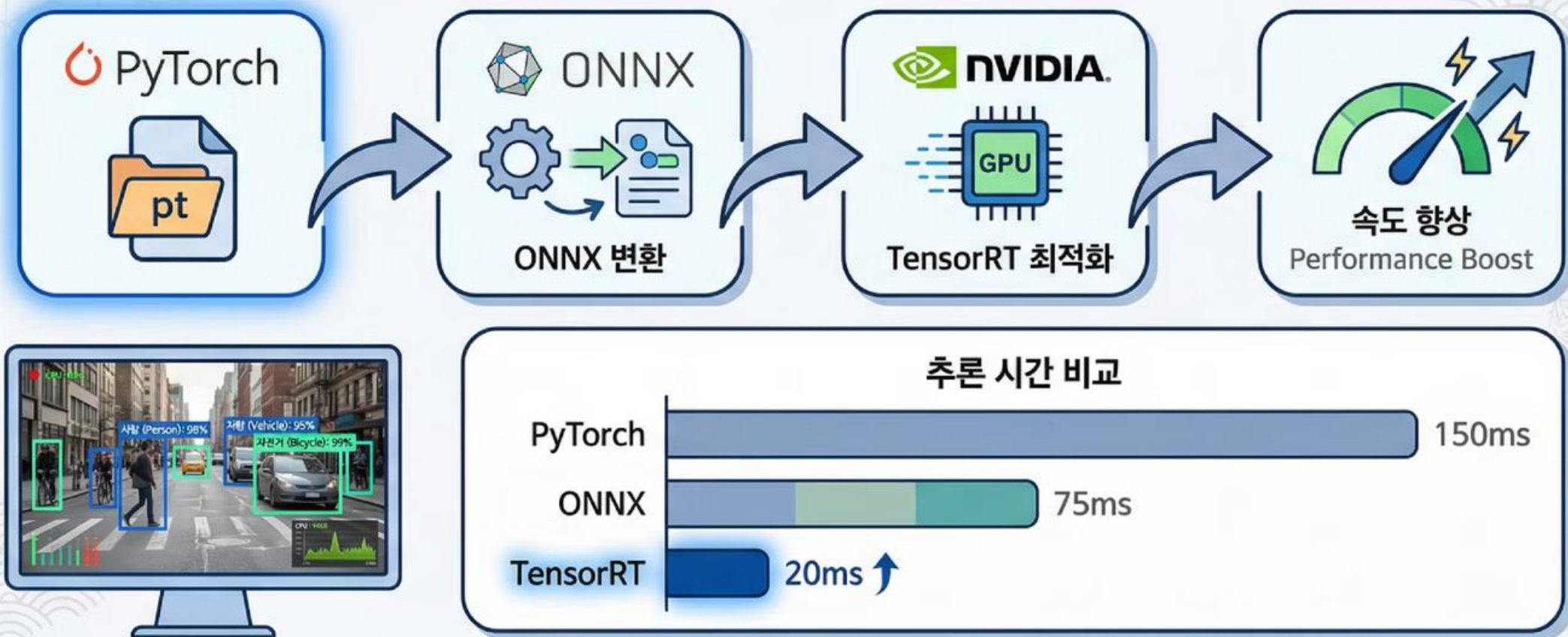
v5n / v8n / v10n (Nano)

- ✓ 리소스 제약: 배터리 소모, 발열, 메모리 한계 극복을 위한 Nano급 모델 필수
- ✓ 경량화 효율: v10n은 CIB 블록으로 적은 연산량 대비 높은 mAP 달성

현장의 제약(리스크·연산·전력)에 맞춘 맞춤형 선택이 모델 성능의 50%를 결정합니다.

실무 배포: 속도 최적화의 완성

PyTorch 모델을 ONNX로 변환하고 TensorRT로 최적화하여 추론 속도를 극대화합니다



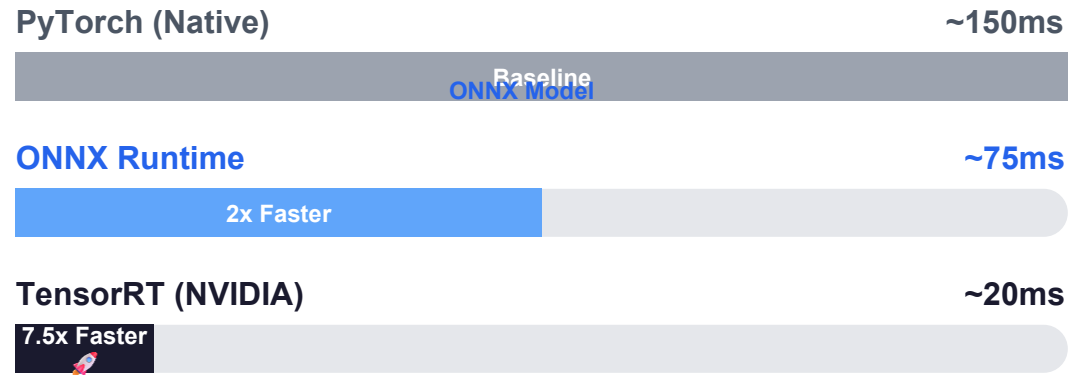
실시간 처리가 필요한 실무 환경에서는 속도 최적화가 성공의 핵심입니다.
모델 변환 과정과 함께 속도 비교 차트, 실시간 객체 검출이 동작하는 모니터 화면

모델 배포 최적화

PyTorch → ONNX → TensorRT

실제 서비스 배포 시, 학습된 PyTorch 모델을 그대로 쓰지 않고
중간 표현(ONNX)을 거쳐 하드웨어 가속 엔진(TensorRT)으로 변환하여 속도를 극대화합니다.

추론 지연 시간(Latency) 비교



형식 변환(ONNX)과 하드웨어 가속(TensorRT) 과정을 통해
추론 지연을 한 자릿수 ms대까지 획기적으로 압축합니다.

💡 핵심 비유: 일반 도로 vs 전용 레이싱 트랙

"범용 차량을 경주용 차로 개조해 트랙을 달린다"

PyTorch가 다양한 길을 갈 수 있는 승용차라면,
TensorRT는 특정 서킷(GPU)에 맞춰 모든 불필요한 부품을 떼어낸 F1 머신입니다.
오직 달리기(추론)만을 위해 최적화됩니다.

YOLO v5



실전 안정성 & 엔지니어링

- CSP + PAN 구조 정립
- Anchor 기반 (Auto Anchor)
- Mosaic 증강으로 소데이터 극복

YOLO v8



Anchor-Free & 구조 혁신

- Anchor-Free (좌표 직접 예측)
- Decoupled Head (분리형 헤드)
- C2f 모듈로 기울기 흐름 개선

YOLO v10



NMS-Free & End-to-End

- NMS-Free (후처리 제거)
- Dual Assignment (중복 억제 학습)
- 효율적 대커널 컨볼루션

√ 꼭 기억할 공식 3개

Precision (정밀도)

$$TP / (TP + FP)$$

"알람이 울렸을 때, 진짜 불이 난 비율"

Recall (재현율)

$$TP / (TP + FN)$$

"실제 불이 났을 때, 알람이 울린 비율"

CIoU Loss

$$IoU - (\rho^2/c^2 + \alpha v)$$

"겹침 + 중심거리 + 비율까지 고려한 정렬"

METAPHOR REMINDER



mAP (평균 정밀도)

"전 과목 평균 점수 — 수학만 잘하고 영어를 못하면 mAP는 낮다"

Focal Loss

"쉬운 문제는 배점 낮게, 어려운 문제에 배점 집중"

Anchor(v5) → Anchor-Free(v8) → NMS-Free(v10)'의 진화 흐름을 이해하는 것이 모델 선택의 핵심입니다.